Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»
(Университет ИТМО)

Факультет программной инженерии и компьютерной техники

Образовательная программа Информатика и вычислительная техника

Направление подготовки (специальность) 09.04.01 Компьютерные системы и технологии

О Т Ч Е Т

Научно-исследовательская работа

Тема задания: Разработка и исследование модели оптимизации сжатия облаков точек на основе анализа существующих методов

Алехин Максим Андреевич, *P4219*

Руководитель практики от университета: Поляков Владимир Иванович, кандидат технических наук, доцент, доцент (квалификационная категория "ординарный доцент") факультета программной инженерии и компьютерной техники, Университета ИТМО.

Практика пройдена с оценкой _____

Подписи членов комиссии:

_____*Ф.И.О.*
(подпись)

_____*Ф.И.О.*
(подпись)

_____*Ф.И.О.*
(подпись)

Дата 23.01.2026

Санкт-Петербург
20 26

Оглавление

Введение 3
1. Анализ предметной области 4
2. Проектирование модели 7
3. Реализация модели 9
4. Тестирование и исследование 15
Заключение 18
Список использованных источников 19
Приложение А. Исходный код программы (main.py) 20
Приложение Б. Отзыв руководителя 25

Введение

В рамках практики была выполнена работа по теме разработки и исследования модели сжатия облаков точек. Основная идея заключалась в том, чтобы собрать понятный и воспроизводимый прототип, который позволяет сравнивать разные методы сжатия по одинаковым правилам. В отчёте фиксируются шаги выполнения работы: от выбора подходов до реализации и проверки на данных. Такой формат важен, потому что даже простые методы могут давать разные результаты в зависимости от настроек и способа измерения.

Цель работы — реализовать и проверить модель сжатия облаков точек на основе известных подходов и экспериментально оценить её поведение. В ходе выполнения были выбраны несколько методов, которые можно использовать как базовые блоки для сжатия геометрии. Для проверки результата применялись измеримые показатели: время выполнения, ошибка восстановления и степень уменьшения объёма данных.

Практическая часть была построена вокруг одной программы, которая загружает исходное облако точек, применяет выбранные методы, сохраняет обработанные версии и автоматически формирует таблицу метрик. Дополнительно была сделана визуальная проверка результатов через Open3D, чтобы можно было быстро сравнить облака в одном окне. Данный проект позволяет добавлять новые методы, менять параметры и повторять эксперименты.

Таким образом, отчёт описывает полный цикл работы: анализ выбранных методов, проектирование схемы обработки, программную реализацию и результаты тестирования.

1. Анализ предметной области

Облако точек это формат хранения трёхмерной геометрии, где объект или сцена представлены как набор точек с координатами x,y,z. Такие данные получают, например, при лидарном сканировании или фотограмметрии, и затем используют для навигации, моделирования сцен. На практике размер облака быстро растёт, потому что детализация напрямую зависит от количества точек. Чем больше точек, тем лучше передаются мелкие элементы поверхности и границы объектов, но это сильно влияет на количество занимаемой памяти и оптимизации самого проекта.

Существует проблема в том, что современные облака точек могут содержать огромное количество точек, и это сразу упирается в объём данных. Передача таких наборов по сети требует высокой пропускной способности, а хранение больших сцен занимает значительное место. При обработке добавляются требования по памяти и времени выполнения, особенно если данные нужны для работы в реальном времени. Ситуация усложняется тем, что облако точек не похоже на изображение: у точек нет фиксированной сетки, а плотность распределения часто неравномерная. В результате обычные методы сжатия «как для картинок» здесь не дают нужной эффективности, и приходится использовать специализированные подходы

Сжатие облаков точек обычно рассматривают в двух режимах: без потерь и с потерями. В режиме без потерь важна точность восстановления, потому что после декодирования данные должны совпасть с исходными. Такой подход подходит для задач, где координаты являются измерением и любое искажение нежелательно. В режиме с потерями допускается небольшое отклонение координат или уменьшение числа точек, но размер данных сокращается сильнее. Этот режим используют там, где важны компактность и скорость, а небольшая ошибка не разрушает смысл сцены. На практике оба режима встречаются в одних и тех же приложениях, потому что требования могут отличаться даже внутри одной системы.

Один из наиболее распространённых способов кодирования геометрии связан с иерархическим разбиением пространства на области, например через октодерево. Здесь идея заключается в том, что вместо хранения всех точек «напрямую» кодируется структура заполнения пространства: где есть точки, а где их нет. Точность при этом связывают с глубиной разбиения, потому что чем меньше итоговый размер ячейки, тем точнее можно восстановить положение точек. Такой метод хорошо работает, когда облако содержит большие области пустого пространства и плотные участки можно описывать локально. При этом расходы на хранение структуры зависят от того, насколько данные неоднородны. Если плотность сильно меняется, дерево может становиться глубже в отдельных местах и это увеличивает объём служебной информации

Другой важный класс методов опирается на пространственную корреляцию точек. Если точки идут в подходящем порядке, координаты соседних точек отличаются незначительно, поэтому вместо абсолютных координат можно хранить разности между ними. Такие разности имеют меньший разброс значений, а значит их проще эффективно кодировать. Но эффективность этого подхода напрямую зависит от порядка точек: при случайной последовательности разности могут быть большими, и выигрыш уменьшается. В практических системах этот метод часто сочетают со структурированием точек по пространственному принципу, чтобы сохранить локальную близость. В таком виде предсказание становится полезным инструментом именно как часть общей схемы сжатия.

Квантование относится к сжатию с потерями и сводится к округлению координат до значений на дискретной сетке с шагом

В результате уменьшается количество возможных координат, данные становятся «грубее», но их проще хранить и кодировать. Ошибка восстановления при квантовании определяется шагом дискретизации: чем больше Δ, тем меньше размер данных и тем выше отклонение координат. Поэтому квантование обычно выбирают как управляемый компромисс между

качеством восстановления и степенью сжатия. На практике этот этап часто используется перед статистическими методами кодирования, потому что после округления значения начинают чаще повторяться. Именно повторяемость делает дальнейшее кодирование заметно эффективнее

Энтропийные методы кодирования применяются на финальной стадии, когда данные уже приведены к форме, удобной для сжатия. Если значения координат, разностей или индексов ячеек часто повторяются, то их можно представить более короткими кодами, чем в обычной записи фиксированной длины. Это позволяет уменьшить итоговый размер без дополнительной потери качества, если входные данные не меняются. Однако такие методы требуют вычислительных затрат на построение модели вероятностей и само кодирование. Поэтому в сравнении методов важно учитывать не только степень сжатия, но и время кодирования и декодирования. На больших сценах разница по времени становится критичной, особенно если данные должны обрабатываться потоково.

Также при работе с облаками точек часто используют упрощение геометрии, когда уменьшают количество точек за счёт удаления избыточных. Например, воксельная фильтрация заменяет группу близких точек одной «представительной» точкой, что уменьшает плотность данных. Это резко сокращает объём и ускоряет вычисления, но неизбежно убирает часть деталей. Чем сильнее упрощение, тем заметнее становятся изменения в тонких структурах и на границах объектов. Поэтому такой метод применяют тогда, когда детальность не является главным требованием, либо когда важнее скорость обработки. В реальных сценариях упрощение часто используется как предварительный шаг перед дальнейшим кодированием

Таким образом, анализ показал, что эффективность сжатия определяется не одним алгоритмом, а сочетанием нескольких идей. Иерархическое представление помогает описывать структуру пространства, предсказание уменьшает разброс значений, квантование снижает количество уникальных координат, а энтропийное кодирование добирает остаточную избыточность.

При этом параметры вроде глубины разбиения и шага квантования напрямую меняют баланс между размером файла и ошибкой восстановления. Поэтому дальнейшая работа логично переходит к проектированию модели, где можно задавать такие параметры и сравнивать результаты по степени сжатия, скорости и точности восстановления.

2. Проектирование модели

На этапе проектирования нужно было собрать не «набор приёмов», а схему, которую можно запустить на одном облаке точек и получить измеримый результат. В материалах по 3D-технологиям видно, что сама форма представления данных влияет на скорость обработки и объём хранения, поэтому в модели нужно заранее знать, что именно мы улучшаем на каждом шаге [3]. В итоге выбрал последовательный подход: сначала делается преобразование, которое уменьшает точность и делает данные проще по структуре, затем убирается избыточность между соседними значениями, а в конце добавляется упаковка, которая уже работает с подготовленными числами.

Основой модели стало квантование координат. По сути координаты «привязываются» к дискретной сетке, и вместо множества уникальных значений появляются повторяющиеся числа, которые легче сжимать. Это нужно в практических сценариях, где облака точек используются для визуализации или моделирования сцен: небольшая погрешность там чаще всего допустима, зато уменьшается нагрузка на хранение и передачу данных [1,5]. Важный момент в проектировании это шаг квантования Δ, потому что именно он определяет, насколько грубо мы округляем координаты и насколько сильно меняется геометрия после восстановления.

Дальше в модель был добавлен блок, который убирает избыточность между точками через хранение разностей. Идея здесь простая: если точки идут в удачном порядке и находятся рядом в пространстве, то разности координат

будут маленькими и более “удобными” для кодирования. Существует проблема в том, что при случайном порядке точек это преимущество пропадает, и разности могут стать такими же хаотичными, как сами координаты. В источнике про оптимизацию порядка вершин для рендеринга подчёркивается, что порядок влияет на эффективность обработки, и эта мысль хорошо ложится на дельта-кодирование: чем лучше локальность, тем выше шанс выиграть по размеру [7]. Поэтому в проекте дельта-кодирование рассматривалось как логичное продолжение квантования, но с пониманием, что оно может требовать дальнейшего улучшения порядка точек.

Отдельно в проектировании был заложен вариант упрощения облака точек, который меняет точность координат и количество точек. Воксельная фильтрация относится именно к этому классу: она уменьшает плотность облака, оставляя от группы близких точек одну»представительную». В статье про downsampling подчёркивается, что такие методы позволяют резко ускорять вычисления и уменьшать объём данных, но ценой потери деталей, и это важно учитывать заранее [6]. Поэтому этот блок был выделен как отдельная ветка модели: мол это не просто сжатие в привычном смысле, а изменение структуры облака.

Чтобы модель можно было сравнивать честно, ещё на этапе проектирования была продумана схема оценки качества и эффективности. Когда число точек сохраняется, логично считать RMSE и PSNR, потому что можно сравнить координаты напрямую и увидеть, насколько мы отклонились от исходного облака. Когда число точек меняется, RMSE уже не отражает реальную картину, поэтому используется Chamfer Distance, которая сравнивает облака как геометрические множества. Параллельно с этим всегда измеряется размер выходного потока после упаковки и время выполнения, потому что в реальных системах важны сколько данных получилось и насколько это быстро работает [3,4].

Таким образом, итоговая модель получилась не как один алгоритм, а как цепочка независимых блоков, которые можно включать и выключать для

сравнения. Базовая ветка строится вокруг квантования и дальнейшего кодирования разностей, а альтернативная ветка даёт сильное уменьшение размера через упрощение облака. Такой формат проектирования удобен именно для практики: он даёт понятный способ экспериментировать с параметрами, проверять поведение методов на одних и тех же данных и делать выводы по степени сжатия, скорости и качеству восстановления [6,7].

3. Реализация модели

На этом этапе была сделана рабочая программная реализация модели сжатия облаков точек и подготовлен код, который можно запускать на реальных данных.

Сначала была реализована загрузка исходного облака точек из файла и перевод в массив float32. Это необходимо, потому что в таком виде можно считать объём исходных данных через nbytes и потом сравнивать его с «сжатым» представлением. После этого была добавлена обратная операция сохранение облака точек обратно в PLY, чтобы можно было получить отдельные файлы после каждого метода и при необходимости открыть их. (Рисунок 1)

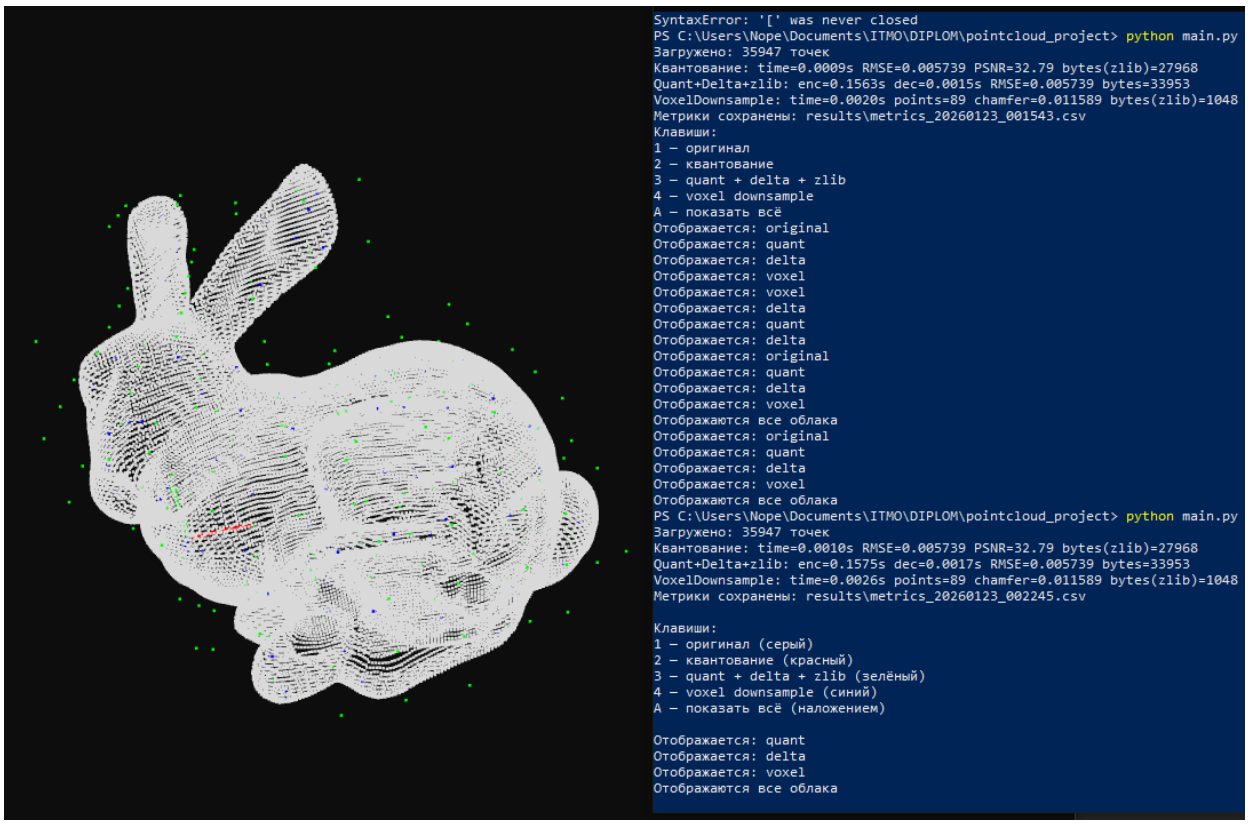


Рисунок 1 – Визуализация облако точек

Ключевые фрагменты кода, которые используются в реализации (полный код приведён в приложении А):

```
pcd = o3d.io.read_point_cloud(path)
points = np.asarray(pcd.points, dtype=np.float32)
raw_bytes = points.nbytes
```

Вся работа строилась вокруг идеи, что один и тот же входной файл должен давать несколько вариантов результата, которые легко сравнить и по метрикам, и визуально.

Дальше были реализованы базовые преобразования, которые относятся к сжатию геометрии. Первое это квантование координат с шагом Δ, где координаты переводятся в целые значения int32, а затем при необходимости восстанавливаются обратно в float. (Рисунок 2)

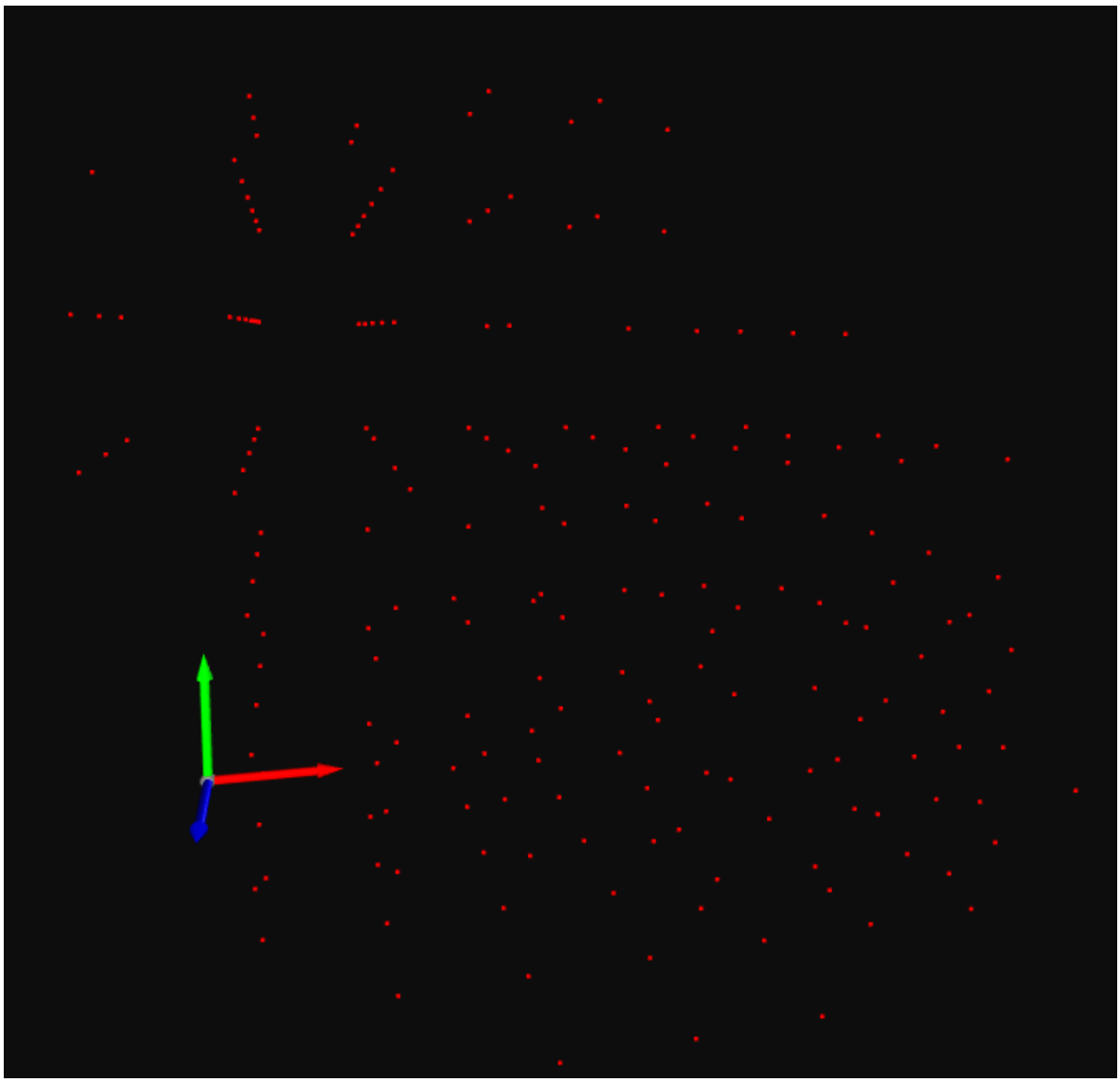


Рисунок 2 – Квантование

Второй метод — дельта-кодирование на квантованных данных, где вместо абсолютных значений хранится разность между соседними точками. Восстановление делается обратной операцией через накопление суммы (cumsum), поэтому метод работает корректно и не добавляет новых потерь по сравнению с квантованием. (Рисунок 3)

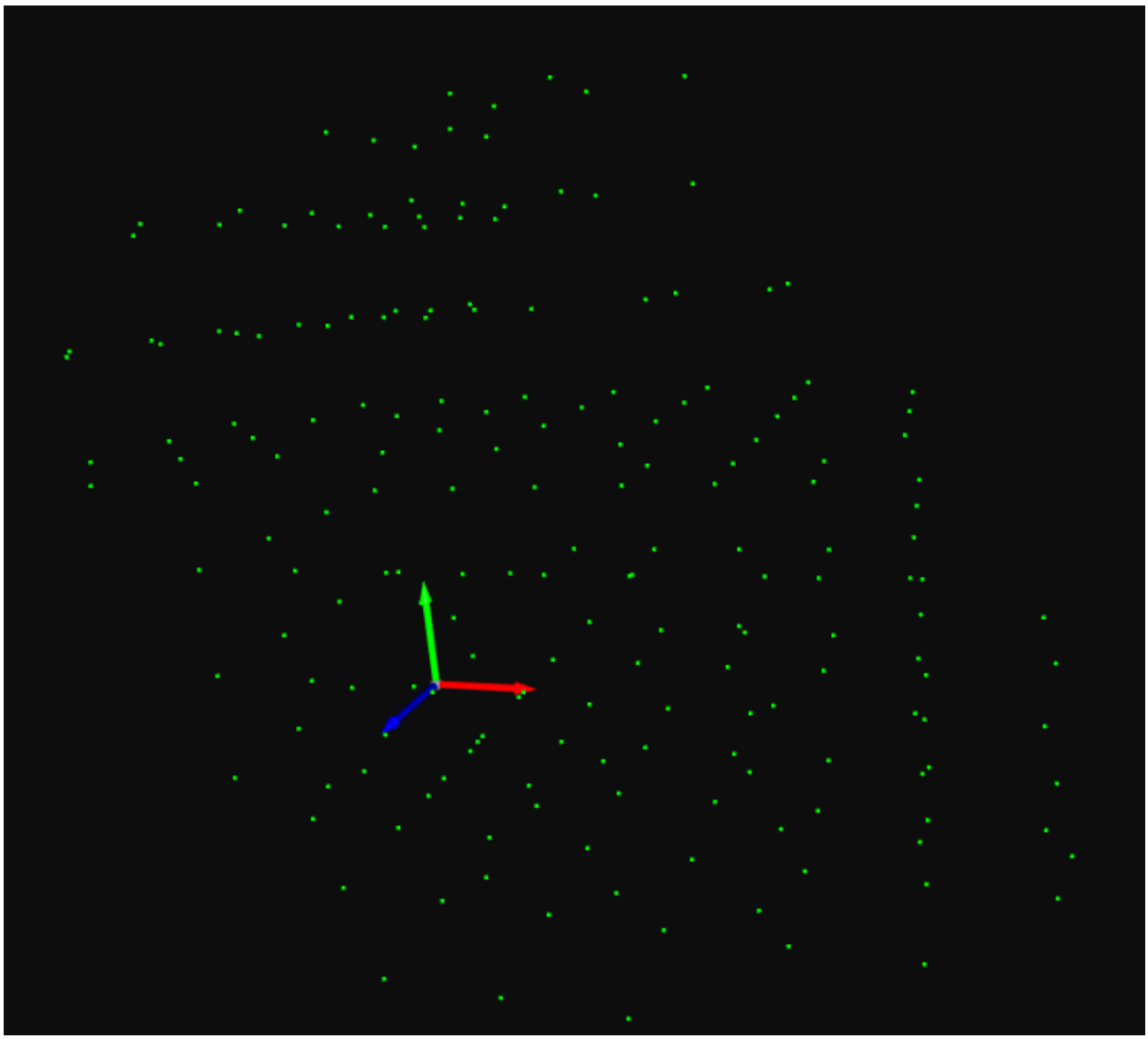


Рисунок 3 – Дельта-кодирование

```
q = np.round(points / step).astype(np.int32)
quant_points = q.astype(np.float32) * step

q_delta = delta_encode_int(q)
q_rest = np.cumsum(q_delta, axis=0).astype(np.int32)
restored = q_rest.astype(np.float32) * step
```

Чтобы получить реальную оценку «степени сжатия», в код был добавлен этап упаковки данных через zlib. Это позволяет получить размер «сжатого» потока в байтах и сравнить его с исходным размером массива координат. Для каждого метода в результате сохранялись значения RawBytes, CompressedBytes и вычислялся коэффициент CompressionRatio. Время

выполнения считалось через perf_counter(), чтобы измерения были стабильнее на коротких операциях. Также отдельно фиксировалось время кодирования и время восстановления.

```
blob = zlib.compress(q.tobytes(order="C"), level=9)
compressed_bytes = len(blob)
compression_ratio = raw_bytes / compressed_bytes
t0 = time.perf_counter()
t1 = time.perf_counter()
```

Для оценки качества результата были добавлены метрики, которые подходят под разные случаи. Для методов, где число точек не меняется, использовался RMSE между исходными координатами и восстановленными, а также PSNR, рассчитанный через диагональ ограничивающего параллелепипеда исходного облака. Для методов, где число точек меняется (например, воксельное упрощение (рисунок 4)), RMSE уже не подходит, поэтому использовалась Chamfer Distance, которая сравнивает облака как геометрические множества. Это сделало сравнение честным: если метод сильно уменьшает количество точек, он всё равно получает оценку качества, но уже по метрике, которая для этого предназначена.

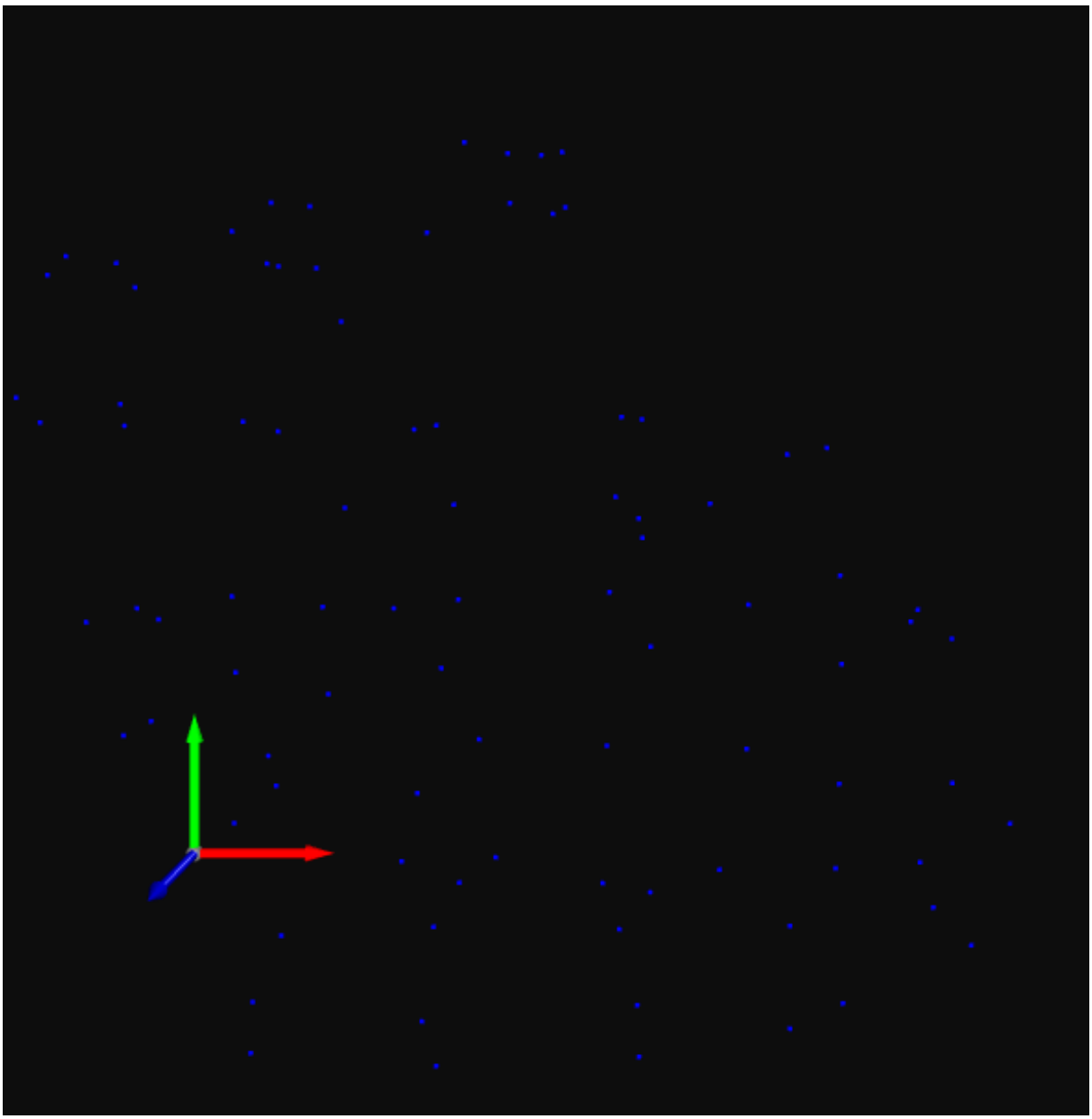


Рисунок 4 – Voxel downsample

```
error = rmse(points, quant_points)
psnr = psnr_by_bbox(points, error)
down = voxel_downsample(points, voxel)
ch = chamfer_distance(points, down)

saved_path = safe_save_metrics_csv(metrics, f"{out_dir}/metrics.csv")
show_visualization(points, quant_points, restored, down)
```

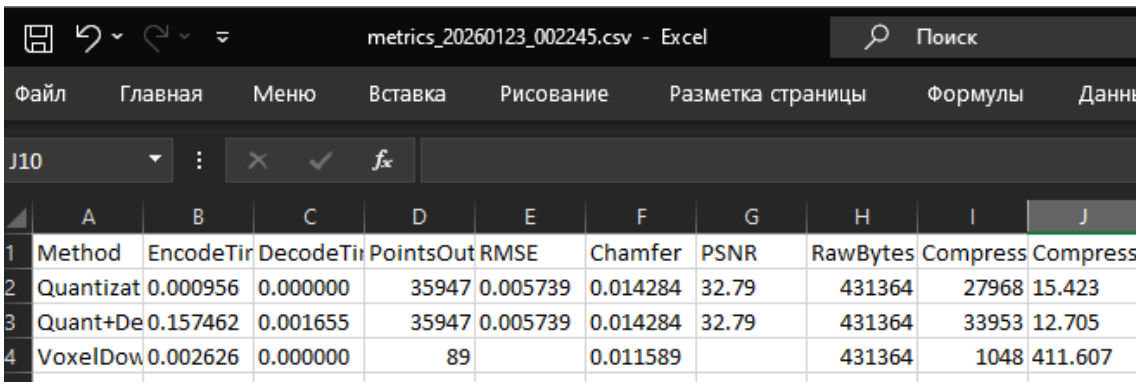
4. Тестирование и исследование

Для проверки реализованных методов сжатия было проведено тестирование на одном облаке точек в формате PLY. Исходный набор содержал 35 947 точек, а объём координат в формате float32 составил 431 364 байта. Сравнение проводилось по трём критериям: время обработки, степень сжатия и ошибка восстановления. В качестве тестовой платформы использовался компьютер под управлением Windows 10 с процессором AMD Ryzen 7 5700X (8 ядер, 3.40 GHz) и объёмом оперативной памяти 64 GB.

В экспериментах сравнивались три варианта обработки. Первый вариант — квантование координат с шагом 0.02, которое снижает точность координат за счёт округления к сетке. Второй вариант — связка квантование + дельта-кодирование + zlib, где после квантования координаты переводятся в разности, а затем упаковываются в сжатый поток. Третий вариант — воксельное упрощение с параметром 0.03, которое уменьшает количество точек за счёт агрегации в пространственных ячейках. Для каждого метода фиксировались время кодирования и восстановления (если оно было), итоговый размер данных в байтах и коэффициент сжатия относительно исходного массива координат.

Метрики качества выбирались так, чтобы сравнение было корректным. Для методов, где число точек не меняется, рассчитывались RMSE и PSNR, которые показывают среднюю ошибку по координатам и качество восстановления относительно размеров сцены. Также использовалась Chamfer Distance, которая подходит и для случаев, когда облака сравниваются как геометрические множества. Для воксельного упрощения RMSE и PSNR не вычислялись, потому что после метода количество точек сильно отличается от исходного, и прямое сравнение «точка к точке» становится некорректным. Поэтому в этом случае основной метрикой качества оставалась Chamfer Distance.

Результаты экспериментов сохранялись в файл metrics.csv, который затем открывался в Excel (Рисунок 5).



	A	B	C	D	E	F	G	H	I	J
1	Method	EncodeTir	DecodeTir	PointsOut	RMSE	Chamfer	PSNR	RawBytes	Compress	Compress
2	Quantizat	0.000956	0.000000	35947	0.005739	0.014284	32.79	431364	27968	15.423
3	Quant+De	0.157462	0.001655	35947	0.005739	0.014284	32.79	431364	33953	12.705
4	VoxelDow	0.002626	0.000000	89		0.011589		431364	1048	411.607

Рисунок 5 – Вывод в csv

Для удобства представления данных в отчёте итоговые значения сведены в таблицу 1.

Таблица 1 – Результат метрики

Метод	Encode Time (c)	Decode Time (c)	Points Out	RMSE	Chamfer	PSNR (дБ)	RawBytes	Compressed Bytes	CompressionRatio
Quantization	0.000956	0.000000	35947	0.005739	0.014284	32.79	431364	27968	15.423
Quant+Delta+zlib	0.157462	0.001655	35947	0.005739	0.014284	32.79	431364	33953	12.705
VoxelDownsample	0.002626	0.000000	89	—	0.011589	—	431364	1048	411.607

По результатам видно, что квантование с шагом 0.02 выполняется практически мгновенно и даёт заметное уменьшение размера данных: итоговый поток после упаковки занимает 27 968 байт, что соответствует сжатию 15.423× при RMSE 0.005739 и PSNR 32.79 дБ. Это означает, что качество восстановления ухудшается умеренно, но объём уменьшается

существенно. Такой метод удобно использовать как базовый и дальше регулировать качество через шаг квантования.

Связка квантование + дельта-кодирование + zlib дала такое же качество восстановления, что и квантование, поскольку сама операция дельта-кодирования выполняется без потерь и не изменяет ошибку квантования. При этом итоговый размер оказался хуже: 33 953 байта, что соответствует сжатию 12.705, а время кодирования существенно выше — 0.157462 сек. Это показывает, что в текущем виде дельта-кодирование не даёт преимущества по объёму и требует доработки, например за счёт улучшенного порядка точек или использования другой схемы кодирования остатков.

Воксельное упрощение показало максимальное уменьшение объёма: итоговый поток составил 1 048 байт, а коэффициент сжатия достиг 411.607×. При этом число точек сократилось до 89, то есть произошло сильное упрощение геометрии. Качество в этом случае оценивалось по Chamfer Distance (0.011589), что позволяет сравнивать облака разного размера. Такой подход полезен там, где допускается потеря мелких деталей ради компактности и скорости, но он уже меняет само облако значительно, поэтому его нельзя напрямую сравнивать по RMSE с методами, которые сохраняют количество точек.

Заключение

В ходе практики была выполнена разработка и проверка прототипа модели сжатия облаков точек. Были реализованы базовые методы обработки координат, организован единый сценарий запуска и подготовлена система автоматического подсчёта метрик. Отдельно были добавлены сохранение результатов в файлы и визуальная проверка через Open3D, чтобы можно было сравнивать не только числа, но и форму облака после восстановления.

Экспериментальная часть показала, что квантование с выбранным шагом даёт хороший компромисс между скоростью работы, степенью уменьшения объёма и точностью восстановления. Связка квантование + дельта-кодирование + zlib в текущем виде не улучшила результат по размеру и оказалась заметно медленнее на этапе кодирования, что говорит о необходимости доработок, если этот подход использовать дальше. Воксельное упрощение обеспечило максимальное уменьшение объёма данных, но достигается это за счёт резкого снижения числа точек и потери детализации, поэтому такой метод подходит только для задач, где допустимо сильное упрощение геометрии.

Таким образом, поставленные задачи были выполнены: изучены и реализованы выбранные методы, проведены измерения времени, степени сжатия и качества восстановления, а результаты оформлены в виде таблицы и выводов. Полученный прототип можно использовать как основу для дальнейших исследований, добавляя новые методы и подбирая параметры сжатия под конкретные требования к скорости, размеру и точности.

Список использованных источников

1. Моделирование виртуальной среды в технологиях кибервизуализации и виртуального присутствия [Электронный ресурс]. Статьи. Режим доступа: https://cyberleninka.ru/article/n/modelirovanie-virtualnoy-sredy-v-tehnologiyah-kibervizualizatsii-i-virtualnogo-prisutstviya, свободный
2. Point Cloud Registration Hybrid Method [Электронный ресурс]. Статьи. Режим доступа: https://ceur-ws.org/Vol-2893/short_10.pdf, свободный
3. 3d-технологии в современных информационных системах: теория и практика [Электронный ресурс]. Статьи. Режим доступа: https://spcras.ru/documents/books/tutorial_kuleshov2021.pdf, свободный
4. Use of point clouds for video surveillance system cover zone imitation [Электронный ресурс]. Статьи. Режим доступа: https://ceur-ws.org/Vol-2344/short3.pdf, свободный
5. Моделирование особенностей ландшафта с помощью облаков точек в системах виртуального окружения [Электронный ресурс]. Статьи. Режим доступа: https://library.keldysh.ru/prep_vw.asp?pid=9988, свободный
6. Exact Point Cloud Downsampling for Fast and Accurate Global Trajectory Optimization [Электронный ресурс]. Статьи. Режим доступа: https://arxiv.org/abs/2307.02948, свободный
7. Rendering Point Clouds with Compute Shaders and Vertex Order Optimization [Электронный ресурс]. Статьи. Режим доступа: https://arxiv.org/abs/2104.07526, свободный
8. Wikipedia [Электронный ресурс]. Статьи. Режим доступа: https://ru.wikipedia.org/wiki/Sparse_Voxel_Octree, свободный

Приложение А. Исходный код программы (main.py)

```
import os
import csv
import time
import zlib
import numpy as np
import open3d as o3d


# ----------------------------
# IO
# ----------------------------

def load_point_cloud(path: str) -> np.ndarray:
    """Загрузка облака точек из PLY и преобразование в NumPy"""
    pcd = o3d.io.read_point_cloud(path)
    return np.asarray(pcd.points, dtype=np.float32)


def save_point_cloud(points: np.ndarray, path: str) -> None:
    """Сохранение облака точек в PLY"""
    os.makedirs(os.path.dirname(path), exist_ok=True)
    pcd = o3d.geometry.PointCloud()
    pcd.points = o3d.utility.Vector3dVector(points.astype(np.float64))
    o3d.io.write_point_cloud(path, pcd)


# ----------------------------
# Метрики
# ----------------------------

def rmse(original: np.ndarray, restored: np.ndarray) -> float:
    """RMSE корректно считается только при одинаковом количестве точек"""
    return float(np.sqrt(np.mean((original - restored) ** 2)))


def chamfer_distance(a_points: np.ndarray, b_points: np.ndarray) -> float:
    """Chamfer Distance через Open3D (подходит для разных размеров
облаков)"""
    a = o3d.geometry.PointCloud()
    b = o3d.geometry.PointCloud()
    a.points = o3d.utility.Vector3dVector(a_points.astype(np.float64))
    b.points = o3d.utility.Vector3dVector(b_points.astype(np.float64))

    d1 = np.asarray(a.compute_point_cloud_distance(b), dtype=np.float32)
    d2 = np.asarray(b.compute_point_cloud_distance(a), dtype=np.float32)
    return float(d1.mean() + d2.mean())


def psnr_by_bbox(original: np.ndarray, error_rmse: float) -> float:
    """PSNR по диагонали bbox (часто используют для 3D-геометрии)"""
    if error_rmse <= 1e-12:
        return float("inf")
    mn = original.min(axis=0)
    mx = original.max(axis=0)
    diag = float(np.linalg.norm(mx - mn))
    return float(20.0 * np.log10(diag / error_rmse))


# ----------------------------
# Методы сжатия
```

```
# ------------------------------

def quantize_to_int(points: np.ndarray, step: float) -> np.ndarray:
    """Квантование: float -> int32"""
    return np.round(points / step).astype(np.int32)


def dequantize_from_int(q: np.ndarray, step: float) -> np.ndarray:
    """Обратное квантование: int32 -> float32"""
    return q.astype(np.float32) * step


def delta_encode_int(q: np.ndarray) -> np.ndarray:
    """Дельта-кодирование для int32 координат"""
    out = np.empty_like(q)
    out[0] = q[0]
    out[1:] = q[1:] - q[:-1]
    return out


def delta_decode_int(d: np.ndarray) -> np.ndarray:
    """Восстановление после дельта-кодирования"""
    out = np.empty_like(d)
    out[0] = d[0]
    out[1:] = np.cumsum(d[1:], axis=0) + out[0]
    return out


def zlib_pack_int32(arr: np.ndarray) -> bytes:
    """Упаковка int32 массива через zlib"""
    raw = arr.tobytes(order="C")
    return zlib.compress(raw, level=9)


def zlib_unpack_int32(blob: bytes, shape) -> np.ndarray:
    """Распаковка int32 массива через zlib"""
    raw = zlib.decompress(blob)
    return np.frombuffer(raw, dtype=np.int32).reshape(shape)


def voxel_downsample(points: np.ndarray, voxel: float) -> np.ndarray:
    """Воксельное упрощение (уменьшает количество точек)"""
    pcd = o3d.geometry.PointCloud()
    pcd.points = o3d.utility.Vector3dVector(points.astype(np.float64))
    down = pcd.voxel_down_sample(voxel_size=voxel)
    return np.asarray(down.points, dtype=np.float32)


# ------------------------------
# CSV (с защитой от блокировки файла)
# ------------------------------

def safe_save_metrics_csv(rows, path: str) -> str:
    """Сохранение метрик в CSV; если файл открыт (Excel), создаёт новый с
таймштампом"""
    os.makedirs(os.path.dirname(path), exist_ok=True)

    base, ext = os.path.splitext(path)
    try_path = path

    for attempt in range(3):
        try:
            with open(try_path, "w", newline="", encoding="utf-8") as f:
                writer = csv.writer(f, delimiter=";")
```

```
                writer.writerow([
                    "Method", "EncodeTime_sec", "DecodeTime_sec",
"PointsOut",
                    "RMSE", "Chamfer", "PSNR", "RawBytes", "CompressedBytes",
"CompressionRatio"
                ])
                for r in rows:
                    writer.writerow(r)
            return try_path
        except PermissionError:
            # если файл заблокирован Excel — пишем в новый
            stamp = time.strftime("%Y%m%d_%H%M%S")
            try_path = f"{base}_{stamp}{ext}"

    return try_path


# ----------------------------
# Визуализация
# ----------------------------

def show_visualization(points, quant_points, restored, down):
    """Окно Open3D с переключением облаков по клавишам"""
    pcd_original = o3d.geometry.PointCloud()
    pcd_original.points =
o3d.utility.Vector3dVector(points.astype(np.float64))
    pcd_original.paint_uniform_color([0.85, 0.85, 0.85])  # светло-серый

    pcd_quant = o3d.geometry.PointCloud()
    pcd_quant.points =
o3d.utility.Vector3dVector(quant_points.astype(np.float64))
    pcd_quant.paint_uniform_color([1.0, 0.2, 0.2])  # красный

    pcd_restored = o3d.geometry.PointCloud()
    pcd_restored.points =
o3d.utility.Vector3dVector(restored.astype(np.float64))
    pcd_restored.paint_uniform_color([0.2, 1.0, 0.2])  # зелёный

    pcd_down = o3d.geometry.PointCloud()
    pcd_down.points = o3d.utility.Vector3dVector(down.astype(np.float64))
    pcd_down.paint_uniform_color([0.2, 0.6, 1.0])  # синий

    cloud_dict = {
        "original": pcd_original,
        "quant": pcd_quant,
        "delta": pcd_restored,
        "voxel": pcd_down,
    }

    vis = o3d.visualization.VisualizerWithKeyCallback()
    vis.create_window(window_name="Point Cloud Compression Demo")
    opt = vis.get_render_option()
    opt.background_color = np.asarray([0.05, 0.05, 0.05])  # тёмный фон
    opt.point_size = 2.0

    current = {"name": "original"}
    vis.add_geometry(cloud_dict[current["name"]])

    def switch(name):
        def _cb(v):
            v.clear_geometries()
            v.add_geometry(cloud_dict[name])
            current["name"] = name
            print(f"Отображается: {name}")
```

```
            return False
        return _cb

    def show_all(v):
        v.clear_geometries()
        for c in cloud_dict.values():
            v.add_geometry(c)
        current["name"] = "all"
        print("Отображаются все облака")
        return False

    vis.register_key_callback(ord("1"), switch("original"))
    vis.register_key_callback(ord("2"), switch("quant"))
    vis.register_key_callback(ord("3"), switch("delta"))
    vis.register_key_callback(ord("4"), switch("voxel"))
    vis.register_key_callback(ord("A"), show_all)

    print("Клавиши: 1 — оригинал, 2 — квантование, 3 — quant+delta, 4 —
voxel, A — всё вместе")
    vis.run()
    vis.destroy_window()


# -----------------------------
# Главная часть
# -----------------------------

if __name__ == "__main__":
    input_path = "data/sample.ply"
    out_dir = "results"
    os.makedirs(out_dir, exist_ok=True)

    points = load_point_cloud(input_path)
    raw_bytes = points.nbytes
    print(f"Загружено: {points.shape[0]} точек")

    metrics = []

    # 1) Квантование
    step = 0.02
    t0 = time.perf_counter()
    q = quantize_to_int(points, step)
    quant_points = dequantize_from_int(q, step)
    t1 = time.perf_counter()

    err = rmse(points, quant_points)
    ps = psnr_by_bbox(points, err)
    ch = chamfer_distance(points, quant_points)
    blob = zlib_pack_int32(q)

    save_point_cloud(quant_points, f"{out_dir}/quant_step_{step}.ply")

    metrics.append([
        "Quantization",
        f"{(t1 - t0):.6f}", "0.000000", len(quant_points),
        f"{err:.6f}", f"{ch:.6f}", f"{ps:.2f}",
        raw_bytes, len(blob), f"{raw_bytes / len(blob):.3f}"
    ])

    # 2) Quant + Delta + zlib
    t0 = time.perf_counter()
    q_delta = delta_encode_int(q)
    blob_delta = zlib_pack_int32(q_delta)
    t1 = time.perf_counter()
```

```
t2 = time.perf_counter()
q_delta_rest = zlib_unpack_int32(blob_delta, q_delta.shape)
q_rest = delta_decode_int(q_delta_rest)
restored = dequantize_from_int(q_rest, step)
t3 = time.perf_counter()

err2 = rmse(points, restored)
ps2 = psnr_by_bbox(points, err2)
ch2 = chamfer_distance(points, restored)

save_point_cloud(restored, f"{out_dir}/quant_delta_zlib_step_{step}.ply")

metrics.append([
    "Quant+Delta+zlib",
    f"{(t1 - t0):.6f}", f"{(t3 - t2):.6f}", len(restored),
    f"{err2:.6f}", f"{ch2:.6f}", f"{ps2:.2f}",
    raw_bytes, len(blob_delta), f"{raw_bytes / len(blob_delta):.3f}"
])

# 3) VoxelDownsample
voxel = 0.03
t0 = time.perf_counter()
down = voxel_downsample(points, voxel)
t1 = time.perf_counter()

ch3 = chamfer_distance(points, down)
blob_down = zlib.compress(down.tobytes(order="C"), level=9)

save_point_cloud(down, f"{out_dir}/voxel_{voxel}.ply")

metrics.append([
    "VoxelDownsample",
    f"{(t1 - t0):.6f}", "0.000000", len(down),
    "—", f"{ch3:.6f}", "—",
    raw_bytes, len(blob_down), f"{raw_bytes / len(blob_down):.3f}"
])

saved_csv = safe_save_metrics_csv(metrics, f"{out_dir}/metrics.csv")
print(f"Метрики сохранены в: {saved_csv}")

show_visualization(points, quant_points, restored, down)
```

Приложение Б. Отзыв руководителя

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»
(Университет ИТМО)

О Т З Ы В

РУКОВОДИТЕЛЯ НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЫ

Обучающийся Алехин М.А.
(Фамилия, И.О.)
Факультет Программной инженерии и компьютерной техники
Группа P4219
Образовательная программа Информатика и вычислительная техника
Направление подготовки (специальность) Компьютерные системы и технологии
Место прохождения практики ИТМО
Должность практиканта Практикант
Тема индивидуального задания Разработка и исследование модели оптимизации основе анализа существующих методов

проходил(а) практику с 15.09 2025 года по 28. 01 2026 года.
Инструктаж по ознакомлению с требованиями охраны труда, техники безопасности, пожарной безопасности, а также правилами внутреннего распорядка проведен в установленном порядке.

Оценка достигнутых результатов

№ п/п	Планируемые результаты обучения (показатели)	Оценка			
		5	4	3	2
1.		+			
2.	Анализ предметной области	+			
3.	Проектирование модели	+			
4.	Реализация модели	+			
5.	Тестирование и исследование	+			
Итоговая оценка					

Отмеченные достоинства:

1. Самостоятельность магистранта в выполнении работы.
2. Инициативность практиканта в достижении поставленной цели.
3. Соблюдение трудовой дисциплины.
4. Содержание и качество оформления отчета.

Отмеченные недостатки: Нет.

Заключение: Задание выполнено полностью и заслуживает оценки «ОТЛИЧНО».

Руководитель практики Поляков В.И.

23. 01 .2025